



Distributed and Parallel Computing Project

Mini-Multiplayer Game

Joker Thief



Team Members

Utkarsh Rana [202251148]

Prashant Bharti [202251102]

Vedulla Sai Vardhan [202251156]

The Game Concept

What is Joker Thief?

- **Concept:** A real-time multiplayer card game.
- **Players:** 2-8 humans or 1 human vs. 2-5 AI bots.
- **Deck:** Standard 52-card deck + 1 Joker (53 cards total).
- **Objective:** Avoid being the last player holding the Joker.
- **The Loser:** The player left with the Joker is "The Thief."



Player

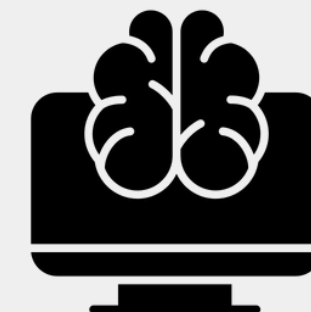


Player

OR



Player



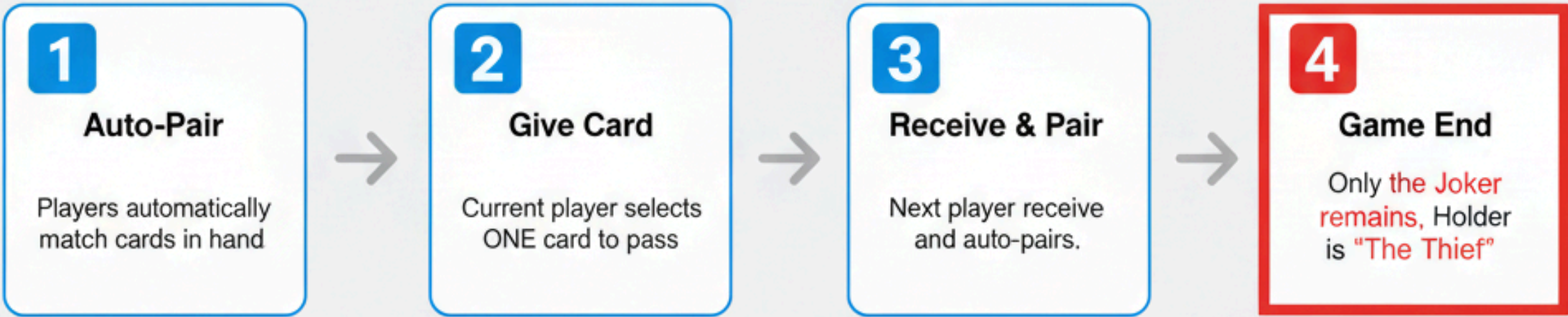
Computer

Core Mechanics & The Twist

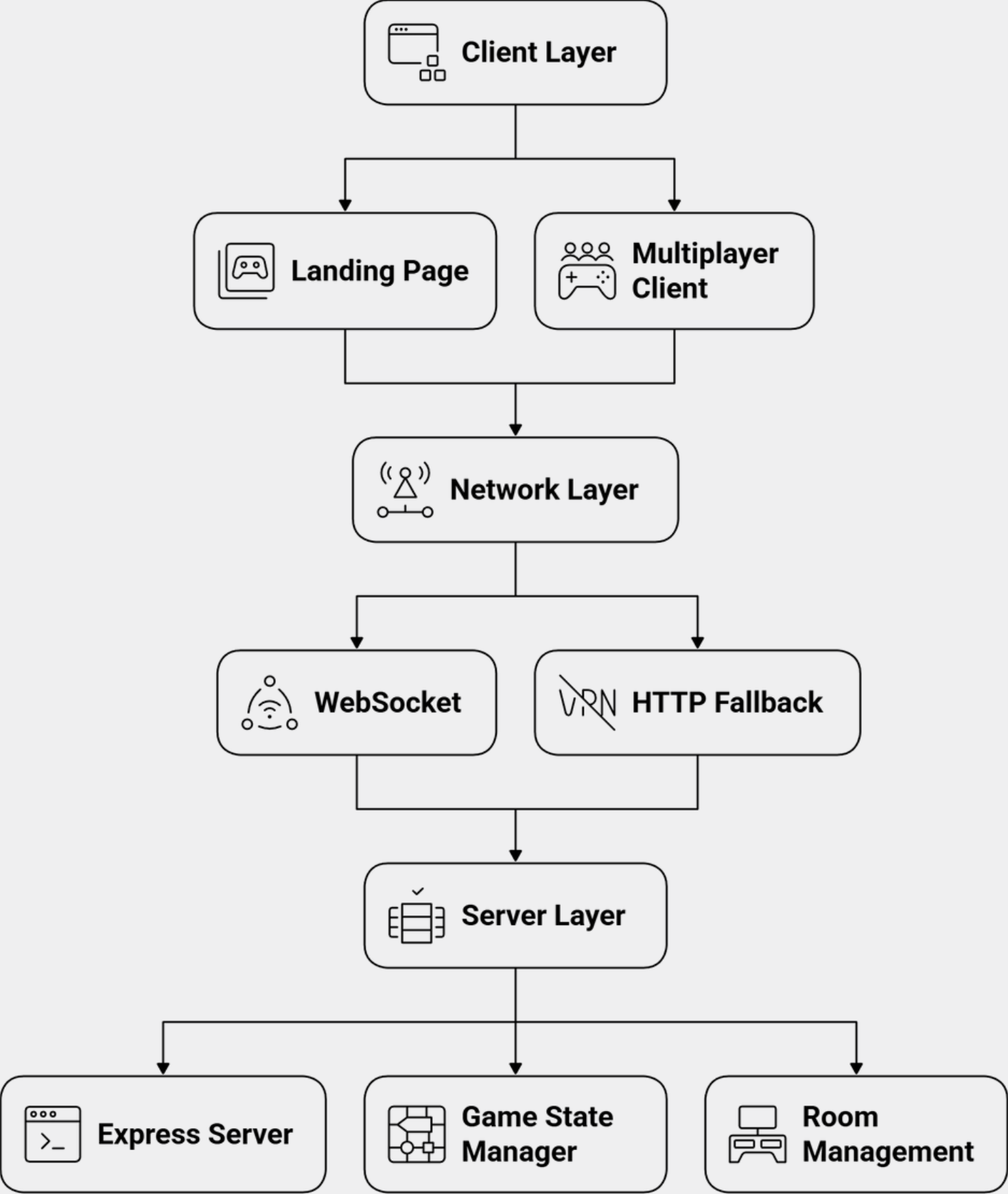
How do you pair cards?



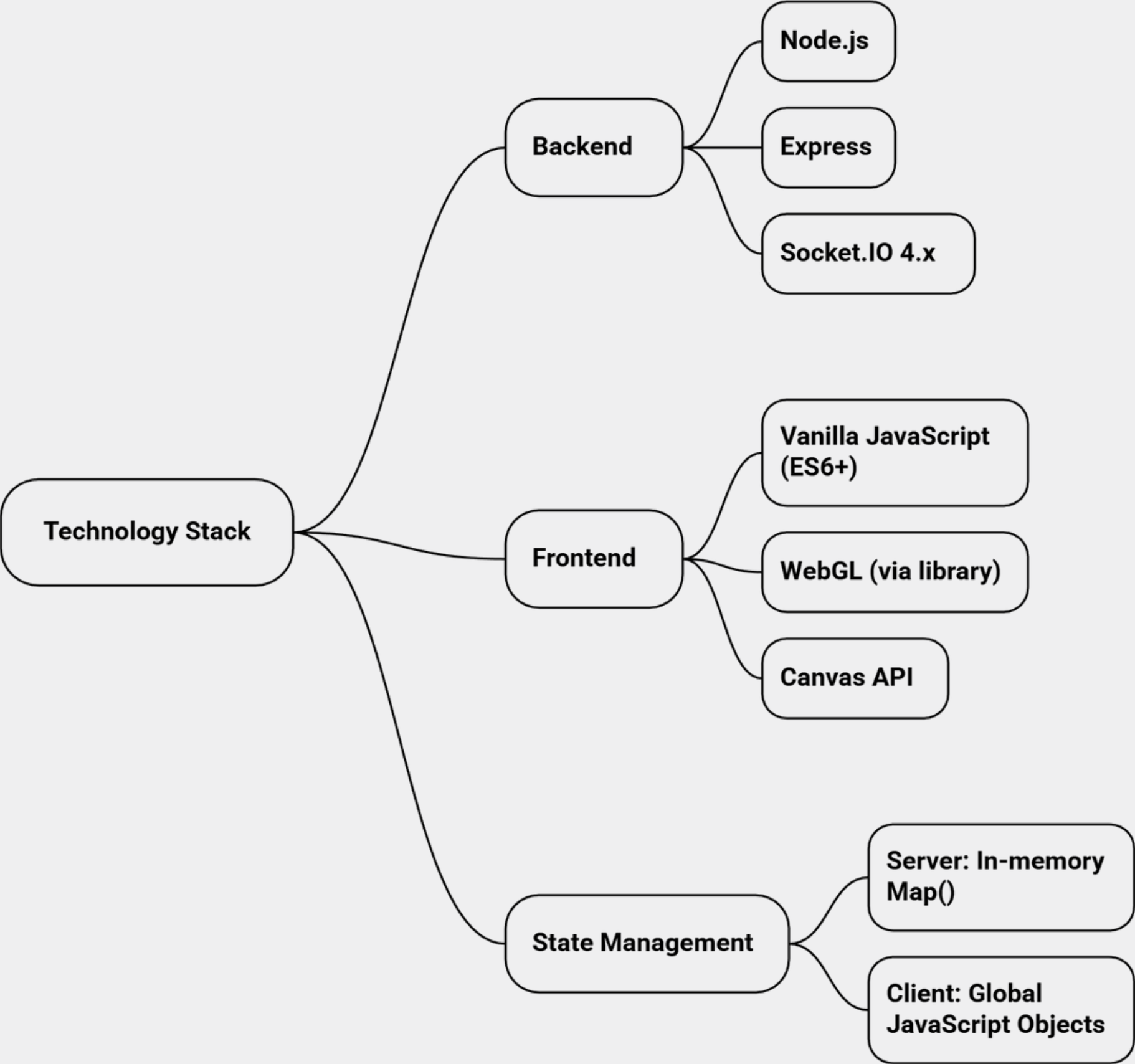
Receiver Flow



System Architecture



Technology Stack Using Websocket



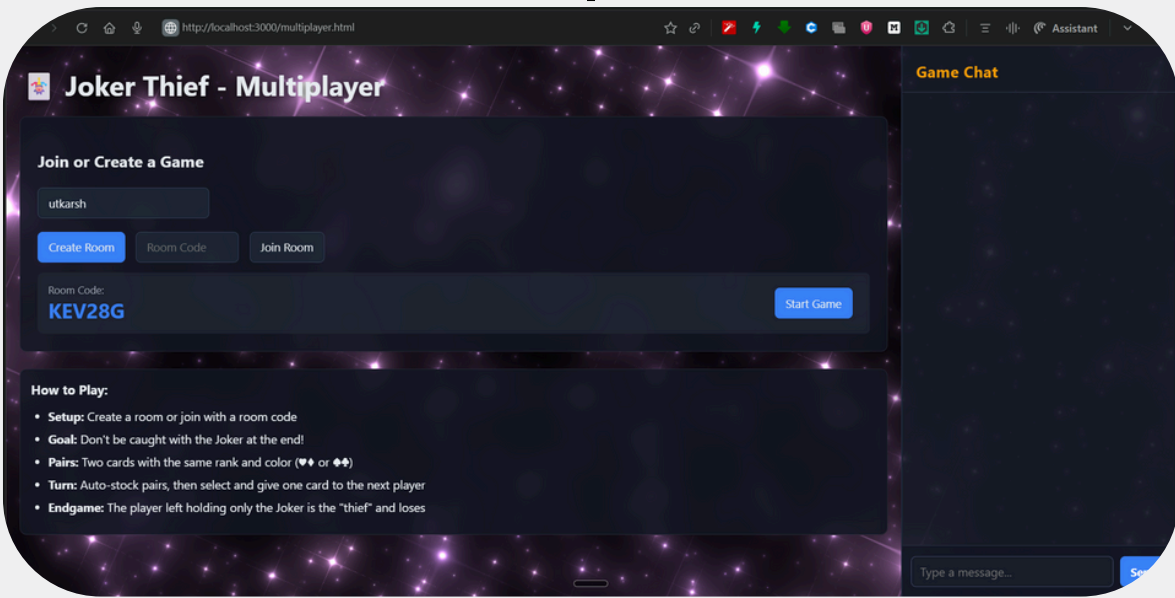
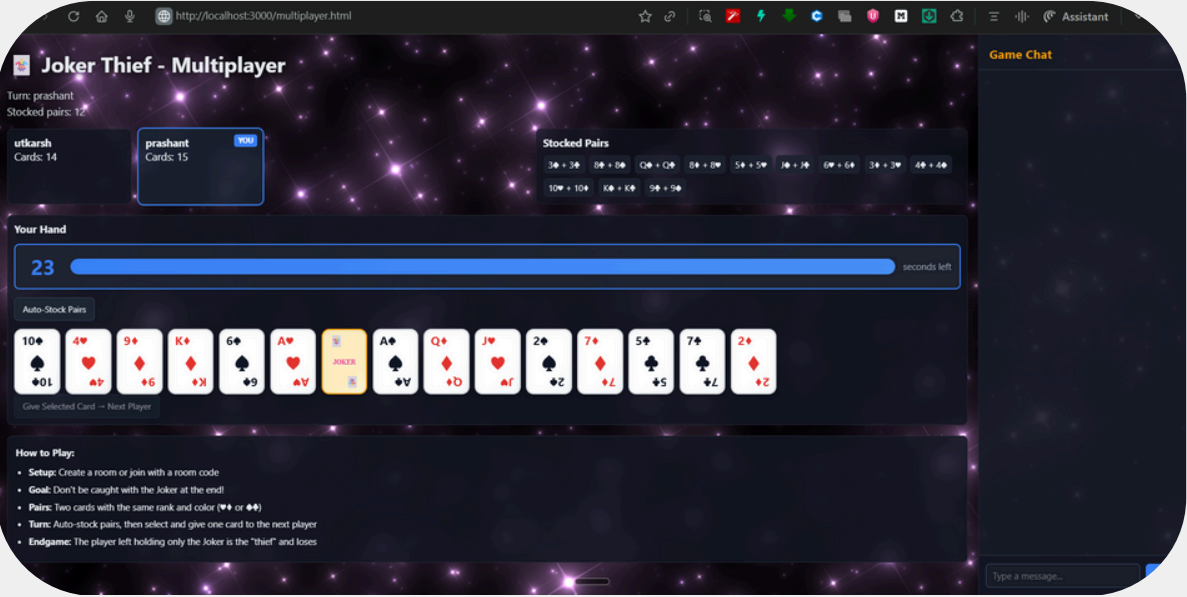
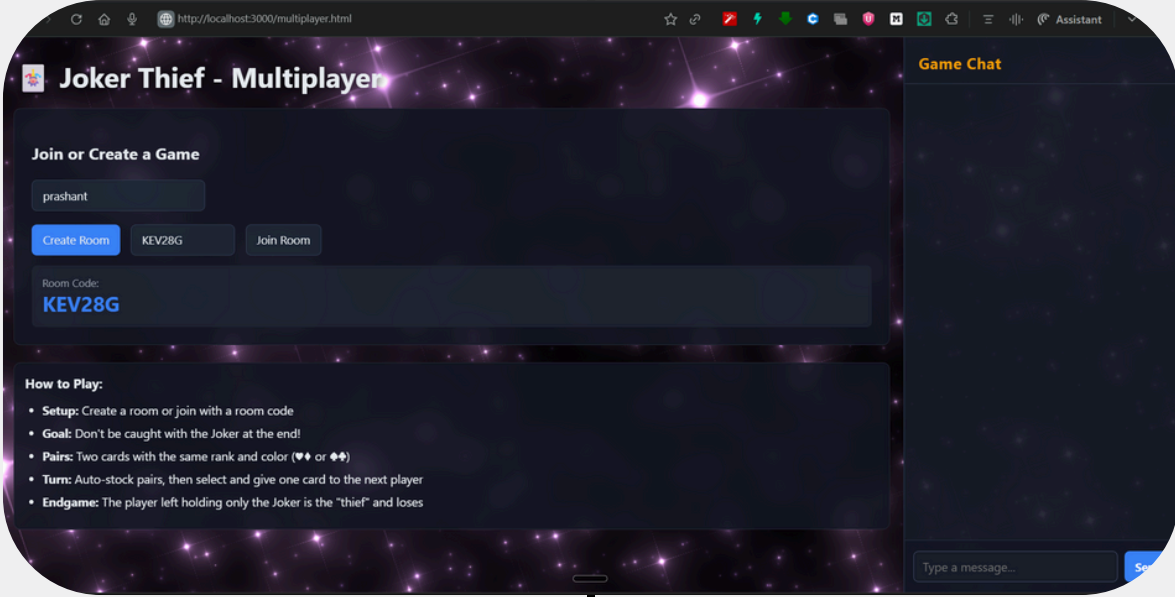
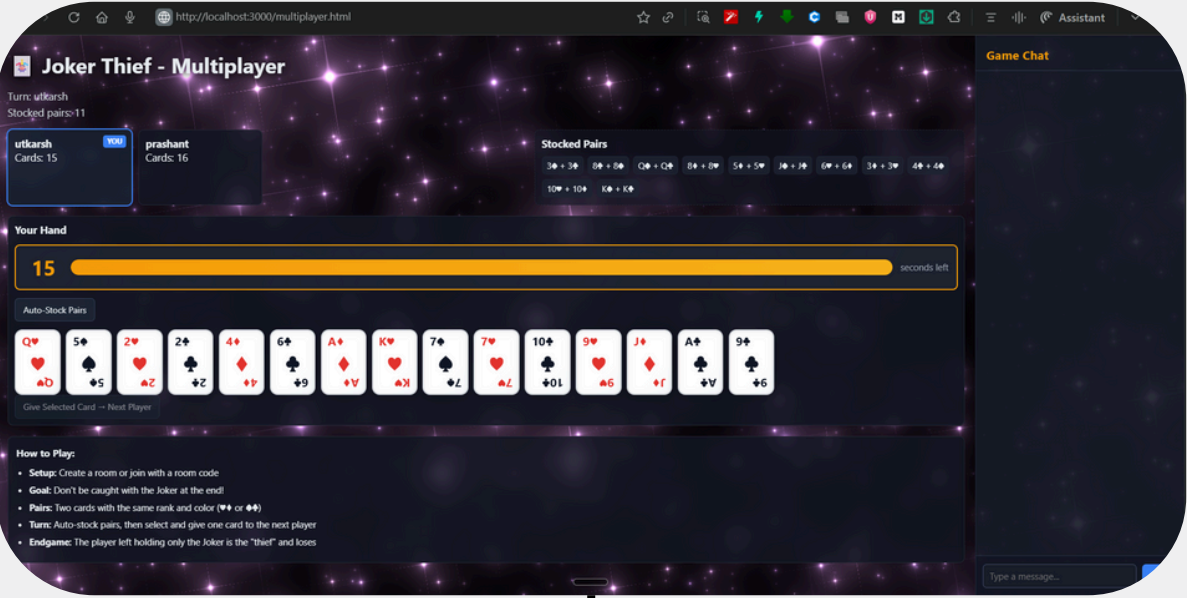
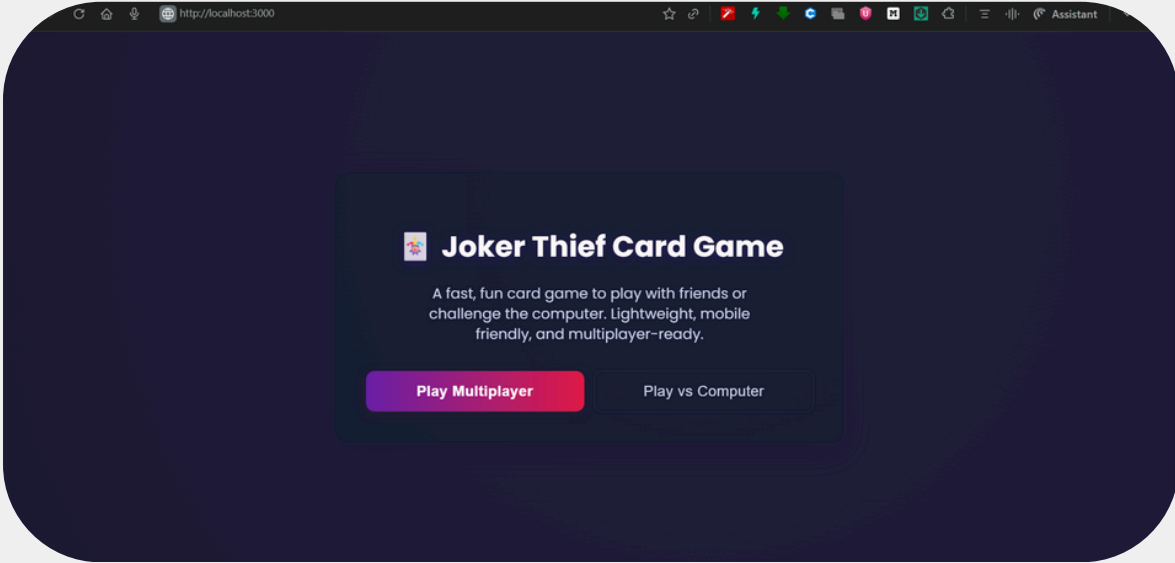
Event-Driven Architecture

Event	Direction	Purpose
create-room	C → S	Initialize a new game room.
room-created	S → C	Confirm creation, send back roomId.
join-room	C → S	Join an existing room.
game-state	S → C	Broadcasts the public game state to all.
start-game	C → S	(Host only) Starts the game.
auto-pair	C → S	(Client-side logic, server validates)
give-card	C → S	The core game action. Pass card.
chat-message	Bi-directional	Real-time chat.

Core Socket Mechanisms

Component	How It Works	Example from Code
Event-Driven Messages	Named events carry JSON data	socket.emit('give-card', {index: 2})
Room Isolation	Each game is a separate "room"	socket.join('ABC123') + io.to('ABC123').emit(...)
Server Authority	All logic runs server-side; clients only request actions	Server validates turn, card index, and updates state
Selective Broadcasting	Public state to all, private hands to individuals	io.emit('game-state') + socket.emit('your-hand')

Chat Box



```
PS C:\Users\utkar\joker-thief-multiplayer> npm run dev
Player connected: SJnKBBq8kT2CFE3MAAAB
Player connected: J1dNUBMIVpXe9GDJAAAD
```

Project Links & Demo

Click the below links

Project Resources

[Live Game](#)